DAY 60 | KAFKA MONITORING | METRICS & LAG | OBSERVABILITY

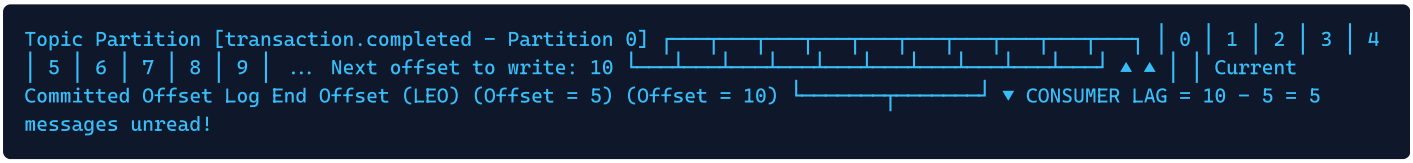
Kafka Monitoring, Topic Metrics & Consumer Lag

Complete Implementation Reference Notes: Log-End-Offset, Consumer Lag Calculation, Kafka UI Dashboard, CLI Inspection, Programmatic AdminClient Lag Monitor & Spring Boot Actuator.

PROJECT	MONITORED TOPICS	CONSUMER GROUP	GENERATED DATE
Digital Banking System	**transaction.completed, beneficiary.activated**	**notification-service-group**	**2026-10-26**

1. Understanding Consumer Lag & Partition Metrics

In a financial banking system, **Consumer Lag** is the single most critical health indicator for Kafka. If consumer lag increases, customers experience delayed OTPs, late transaction balance alerts, or delayed fraud warnings.



Key Terminology

Term	Definition	Significance in Banking
Log End Offset (LEO)	The offset of the next message to be produced into the partition.	Shows total volume of transactions produced by the banking backend.
Current Offset	The last offset successfully read and committed by the consumer group.	Shows how far the `notification-service` has progressed.
Consumer Lag	`Lag = LEO - Current Offset`	• **Lag = 0**: Healthy real-time consumption. • **Lag growing**: Consumer is falling behind (slow SMTP, DB bottlenecks). • **Lag constant > 0**: Consumer has halted or crashed.

2. The 4 Ways to Monitor Kafka in Our Project

TIER 1 **Visual Web UI (Kafka UI)**

Already configured in `docker-compose.yaml` on port **8085**. Provides point-and-click visibility into topics, partitions, consumer group lag, and message contents.

CLI Diagnostics (Docker Exec)

Built-in Apache Kafka shell scripts running directly inside the `banking-kafka` container. Perfect for fast terminal diagnostics and emergency offset resets.

TIER 3 **Programmatic Lag Monitor**

Custom Spring Service using Kafka's `AdminClient` API to compute consumer lag dynamically and alert when lag crosses a safety threshold.

TIER 4 **Spring Boot Actuator & Prometheus**

Exposing Kafka consumer metrics through Micrometer for Prometheus scraping and Grafana dashboards (connects to Days 63-65).

3. Tier 1: Visual Monitoring via Kafka UI

Your `docker-compose.yaml` already spins up `provectuslabs/kafka-ui:latest` bound to `http://localhost:8085`.

Step-by-Step Inspection Checklist

1. **Inspect Topics & Partitions:** Open `http://localhost:8085` → Click **Topics** → Select `transaction.completed`.
   - Verify partition count (3 partitions defined in docker-compose).
   - Inspect **Messages In Per Sec** and total message count.
   - Click **Messages** tab to view live JSON payloads and headers.
2. **Inspect Consumer Groups & Lag:** Click **Consumers** on the left sidebar → Select `notification-service-group`.
   - Inspect **Total Lag**: Should stay at 0 or small single digits during active transactions.
   - Inspect per-partition breakdown:

Topic	Partition	Consumer ID	Host	Current Offset	End Offset	Lag
`transaction.completed`	0	notification-1-abc...	/172.20.0.1	42	42	0
`transaction.completed`	1	notification-1-abc...	/172.20.0.1	35	35	0
`transaction.completed`	2	notification-1-abc...	/172.20.0.1	50	50	0

4. Tier 2: Kafka CLI Diagnostic Commands

Run these commands directly in PowerShell / terminal against the Docker container:

4.1 Inspect Consumer Group Lag

```
# Describe consumer group, offsets, and real-time lag across all partitions
docker exec -it banking-kafka /opt/kafka/bin/kafka-consumer-groups.sh \
  --bootstrap-server localhost:9092 \
  --describe \
  --group notification-service-group
```

Expected Output:

```
GROUP                   TOPIC                 PARTITION  CURRENT-OFFSET  LOG-END-OFFSET  LAG
CONSUMER-ID     HOST
notification-service-gr  transaction.completed 0          120             120             0
consumer-1-xxx  /172.20.0.1
notification-service-gr  transaction.completed 1          105             105             0
consumer-1-xxx  /172.20.0.1
notification-service-gr  transaction.completed 2          98              98              0
consumer-1-xxx  /172.20.0.1
```

4.2 Inspect Topic Partition Metadata

```
# Check topic details: partitions, replication factor, ISR (in-sync replicas)
docker exec -it banking-kafka /opt/kafka/bin/kafka-topics.sh \
  --bootstrap-server localhost:9092 \
  --describe \
  --topic transaction.completed
```

4.3 Emergency Offset Reset (Runbook Command)

If an unhandled bug corrupts consumer state and you need to replay messages or skip to latest:

```
# IMPORTANT: Stop notification-service first before resetting offsets!
# Reset offset to earliest (replay all past messages):
docker exec -it banking-kafka /opt/kafka/bin/kafka-consumer-groups.sh \
  --bootstrap-server localhost:9092 \
  --group notification-service-group \
  --topic transaction.completed \
  --reset-offsets --to-earliest --execute

# Or reset to latest (skip all accumulated backlog):
docker exec -it banking-kafka /opt/kafka/bin/kafka-consumer-groups.sh \
  --bootstrap-server localhost:9092 \
  --group notification-service-group \
  --topic transaction.completed \
  --reset-offsets --to-latest --execute
```

5. Tier 3: Programmatic Lag Monitoring in Spring Boot

For custom alerts or health checks, you can query consumer lag in Java using Kafka's official `AdminClient`.

5.1 AdminClient Bean Configuration

notification-service / config / KafkaAdminConfig.java — Java

```java
package org.example.notificationservice.config;

import org.apache.kafka.clients.admin.AdminClient;
import org.apache.kafka.clients.admin.AdminClientConfig;
import org.springframework.beans.factory.annotation.Value;
import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;

import java.util.HashMap;
import java.util.Map;

@Configuration
public class KafkaAdminConfig {

    @Value("${spring.kafka.bootstrap-servers}")
    private String bootstrapServers;

    @Bean
    public AdminClient adminClient() {
        Map<String, Object> configs = new HashMap<>();
        configs.put(AdminClientConfig.BOOTSTRAP_SERVERS_CONFIG, bootstrapServers);
        return AdminClient.create(configs);
    }
}
```

5.2 Consumer Lag Monitoring Service

notification-service / service / KafkaLagMonitorService.java — Java

```java
package org.example.notificationservice.service;

import lombok.RequiredArgsConstructor;
import lombok.extern.slf4j.Slf4j;
import org.apache.kafka.clients.admin.*;
import org.apache.kafka.clients.consumer.OffsetAndMetadata;
import org.apache.kafka.common.TopicPartition;
import org.springframework.scheduling.annotation.Scheduled;
import org.springframework.stereotype.Service;

import java.util.*;
import java.util.concurrent.TimeUnit;

@Service
@RequiredArgsConstructor
@Slf4j
public class KafkaLagMonitorService {

    private final AdminClient adminClient;
    private static final String CONSUMER_GROUP = "notification-service-group";
    private static final long LAG_WARNING_THRESHOLD = 50L;

    // Day 60: Periodically checks consumer lag every 30 seconds
    @Scheduled(fixedRate = 30000)
    public void monitorConsumerLag() {
        try {
            Map<TopicPartition, Long> lagMap = calculateLag(CONSUMER_GROUP);
            long totalLag = lagMap.values().stream().mapToLong(Long::longValue).sum();

            if (totalLag > LAG_WARNING_THRESHOLD) {
                log.warn("⚠️ HIGH CONSUMER LAG DETECTED! Group: '{}', Total Lag: {} messages. Details: {}",
                        CONSUMER_GROUP, totalLag, lagMap);
            } else {
                log.info("✅ Kafka Consumer Lag healthy. Group: '{}', Total Lag: {}", CONSUMER_GROUP, totalLag);
            }
        } catch (Exception ex) {
            log.error("Failed to retrieve Kafka consumer lag metrics", ex);
        }
    }

    public Map<TopicPartition, Long> calculateLag(String groupId) throws Exception {
        // 1. Retrieve current committed offsets for the consumer group
        ListConsumerGroupOffsetsResult groupOffsetsResult = adminClient.listConsumerGroupOffsets(groupId);
        Map<TopicPartition, OffsetAndMetadata> consumerOffsets =
                groupOffsetsResult.partitionsToOffsetAndMetadata().get(5, TimeUnit.SECONDS);

        // 2. Query log end offsets (LEO) for the same partitions
        Map<TopicPartition, OffsetSpec> requestEndOffsets = new HashMap<>();
        for (TopicPartition tp : consumerOffsets.keySet()) {
            requestEndOffsets.put(tp, OffsetSpec.latest());
        }

        Map<TopicPartition, ListOffsetsResult.ListOffsetsResultInfo> endOffsets =
                adminClient.listOffsets(requestEndOffsets).all().get(5, TimeUnit.SECONDS);

        // 3. Compute: Lag = EndOffset - CurrentOffset
        Map<TopicPartition, Long> lagPerPartition = new HashMap<>();
        for (Map.Entry<TopicPartition, OffsetAndMetadata> entry : consumerOffsets.entrySet()) {
            TopicPartition tp = entry.getKey();
            long currentOffset = entry.getValue() != null ? entry.getValue().offset() : 0L;
            long logEndOffset = endOffsets.containsKey(tp) ? endOffsets.get(tp).offset() : 0L;
            long lag = Math.max(0L, logEndOffset - currentOffset);
            lagPerPartition.put(tp, lag);
        }

        return lagPerPartition;
    }
}
```

6. Tier 4: Spring Boot Actuator Kafka Metrics (Day 63/64 Bridge)

Spring Boot can automatically export Kafka metrics into Micrometer / Actuator endpoints.

6.1 Enable Micrometer Kafka Metrics in Properties

application.properties

```
# Expose Actuator metrics endpoints
management.endpoints.web.exposure.include=health,info,metrics,prometheus
management.endpoint.metrics.enabled=true

# Auto-register Kafka Producer and Consumer metrics with Micrometer
spring.kafka.consumer.properties.micrometer.core.kafka.enabled=true
spring.kafka.producer.properties.micrometer.core.kafka.enabled=true
```

6.2 Key Actuator Endpoints for Kafka Metrics

Endpoint URL	Metric Name	Description
`/actuator/metrics/kafka.consumer.fetch.manager.records.lag.max`	`records.lag.max`	The maximum lag across any partition assigned to this consumer instance.
`/actuator/metrics/kafka.consumer.fetch.manager.records.consumed.total`	`records.consumed.total`	Cumulative count of messages processed since consumer startup.
`/actuator/metrics/kafka.producer.record.send.total`	`record.send.total`	Total messages produced by banking app.
`/actuator/metrics/kafka.producer.record.error.total`	`record.error.total`	Count of failed producer transmissions.

7. Hands-On Test: How to Simulate and Verify Consumer Lag

> **Verification Goal:** Intentionally generate lag to observe the metrics in Kafka UI and logs, then let the consumer catch up.

1. **Simulate a Slow Consumer:** In `TransactionEventConsumer.java`, temporarily add a delay:

```
// Temporarily simulate slow processing / slow event handling
Thread.sleep(3000);
```

2. **Produce Multiple Transactions:** Send 10 rapid deposit requests from Postman or a loop:

```
POST http://localhost:8080/api/v1/transactions/deposit
```

3. **Observe Lag Spike:**
   - In Kafka UI (`http://localhost:8085/ui/clusters/local-banking-cluster/consumers/notification-service-group`): watch **Total Lag** climb to **10**.
   - In terminal: run `docker exec -it banking-kafka /opt/kafka/bin/kafka-consumer-groups.sh --bootstrap-server localhost:9092 --describe --group notification-service-group`.
   - In logs: watch `KafkaLagMonitorService` output `⚠️ HIGH CONSUMER LAG DETECTED`.
4. **Watch Recovery:** Remove the `Thread.sleep(3000)`. Within seconds, lag drains back to **0**.

8. Troubleshooting Kafka Lag Issues in Production

Symptom	Root Cause	Remedy
Lag monotonically increasing	Production rate exceeds consumption rate (e.g. 500 tx/sec produced, consumer only handles 100/sec).	• Scale consumers (increase partition count and add consumer instances). • Increase consumer concurrency (`factory.setConcurrency(3)`).
Lag stuck on one partition	Poison pill message throwing unhandled exception repeatedly without DLT.	Apply **Day 58** `@RetryableTopic` or `DeadLetterPublishingRecoverer`.
Frequent Rebalance Storms	Consumer processing takes longer than `max.poll.interval.ms` (default 300s). Broker thinks consumer died.	• Reduce `max.poll.records` (e.g. from 500 to 50). • Increase `max.poll.interval.ms`.